

智能任务协同看板项目

团队协同研发基本盘手册 (含云数据库配置)

文档目的： 本手册整合了项目初始化的全部基础设施配置，包括 GitHub 克隆、智能体(Agent)设置、技能(Skill)矩阵、模型上下文协议(MCP) 以及 Neon 云数据库的接入指南。请团队成员严格按照本手册完成本地沙箱环境的搭建。

1. 源码克隆与环境初始化

全员必须在本地准备好运行环境，并拉取包含所有基础设施配置的代码仓库。

1.1 克隆代码库

```
git clone https://github.com/wangshun/Dashboard_System.git
cd Dashboard_System
```

1.2 环境依赖与工具链安装

确保本地 Node.js 版本 $\geq v20.19.0$ 。全局安装 OpenSpec 核心引擎：

```
npm install -g @fission-ai/openspec@latest
openspec --version
```

2. 云数据库 (Neon) 接入配置

本项目采用 Neon 云数据库并启用分支隔离，防止测试数据污染开发数据。

2.1 数据库连接地址 (URL)

- 开发库 (`main` 分支)：用于日常研发 Agent 写代码与前端联调。
- 沙箱库 (`test-sandbox` 分支)：专门给测试 Agent 执行 Playwright 与重置数据的靶场。

2.2 业务代码接入 (供研发 Agent 使用)

在 `src/main/resources/application.yml` 中配置开发库数据源：

```
spring:
  datasource:
    # 填入 main 分支的地址 (注意 jdbc 前缀)
    url: jdbc:postgresql://ep-xxx-xxx.region.aws.neon.tech/neondb
    username: neondb_owner
    password: <团队共享密码>
    driver-class-name: org.postgresql.Driver
```

2.3 环境变量配置 (供 MCP 与测试 Agent 使用)

 **极度危险：** 必须配置为 ``test-sandbox`` 分支的 URL。若错配为 main 分支，自动化测试将清空开发数据！

四位成员必须在各自操作系统的环境变量中添加：

- 变量名： `DATABASE_URL`
- 变量值： `postgresql://neondb_owner:密码@ep-yyy-yyy.region.aws.neon.tech/neondb?sslmode=require` (沙箱分支地址)

3. 智能体 (Agent) 与技能 (Skill) 矩阵

所有的配置均已固化在项目的 `.trae/` 与 `.agents/` 目录下。

智能体角色	挂载核心能力 (Skills)	核心红线约束
产品设计 Agent	<code>EARS_Requirement</code> , <code>Prototype</code> , <code>openspec-propose</code>	绝对禁止写代码。提议阶段仅允许输出 Markdown 格式的 PRD 与 <code>tasks.md</code> 。
研发 Agent	<code>PostgreSQL_Design</code> , <code>PortDesign</code> , <code>SpringVue_Coding</code>	在 SOLO 模式下，必须严格按照 <code>tasks.md</code> 顺序单步执行并标记完成。
测试 Agent	<code>testcase-xmind-generator</code> , <code>IntegrationTestSkill</code>	执行端到端测试，捕获错误堆栈打回研发。全绿方可执行归档。

4. 物理操作通道 (MCP) 清单

在 `mcp.json` 中已配置五大通道，赋予 Agent 读写与测试能力：

- `project-filesystem`： 仅限当前项目目录的代码文件读写。
- `github-mcp`： 依托 `GITHUB_TOKEN` 自动管理分支与 PR。

- `figma-mcp`：依托 `FIGMA_TOKEN` 读取 UI 设计变量。
- `playwright-mcp`：调度无头浏览器进行页面交互验证。
- `postgres-mcp`：依托系统变量 `DATABASE_URL` 直连沙箱数据库。

⚠️ 密钥安全声明： `mcp.json` 内的任何 Token 与密码均采用 `${VAR}` 变量占位符，严禁直接明文写入并 Push 进 GitHub。所有成员必须在本地配置相应的系统环境变量。